



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A' Grade

CYBER SECURITY

Case Study Report (Unit 4)

AI-Powered Detection and Automated Response to Malicious PowerShell Attacks Using Machine Learning

Group No.: G53

Topic Selected: AI-Powered Detection and Automated Response to Malicious PowerShell
Attacks Using Machine Learning

Submitted by:

Kushagra

23070122122

Submitted to: Dr. Pooja Bagane

Department of Computer Science and Engineering
Symbiosis Institute of Technology, Pune

October 2026

Abstract

PowerShell is an automation tool that is available in all versions of Windows. The reason why adversaries use PowerShell is its reliability, features, and execution of code in memory without writing to the disk. This case study examines how PowerShell has been used in real attacks, including the Odinaff attacks on banks, the POSHSPY backdoor of the APT29 group, the Emotet malware campaigns and the REvil ransomware attack through Kaseya VSA. These cases show that traditional signature-based antivirus finds it hard to catch such scripts, because attackers can easily change the text of a script while keeping its behaviour the same. Based on the lessons from these cases, we built and tested a machine-learning system that reads a PowerShell script, decides whether it is malicious, and takes an automatic response according to its risk. Using the public MPSD dataset, from which more than half of the malicious files had to be removed as duplicates, an ensemble model achieved an F1-score of 0.972, detecting 96.2% of malicious scripts while wrongly flagging only 0.7% of benign scripts. In a test on unseen scripts, the automatic response system contained 91.2% of malicious scripts and wrongly blocked only 0.23% of benign ones. The study shows that explainable machine learning combined with a careful, human-supervised response policy can help organisations defend against PowerShell attacks.

Keywords: PowerShell, fileless malware, case study, machine learning, malware detection, automated incident response, ransomware, MITRE ATT&CK.

1 Introduction

PowerShell is a command-line shell and scripting language made by Microsoft. It is installed by default on Windows and is used by system administrators to automate everyday tasks such as managing users, installing software and changing settings. Because it is powerful and already trusted on the system, attackers also like to use it. With a single PowerShell command, an attacker can download harmful code from the internet and run it directly in memory, without saving any file on the disk. This type of attack is called a *fileless* or *living-off-the-land* attack, because the attacker uses tools that are already present on the computer [1].

PowerShell attacks remain a serious and current problem. Symantec reported that 95.4% of the PowerShell scripts submitted to its analysis service were malicious [2]. MITRE ATT&CK, a widely used knowledge base of attacker behaviour, lists PowerShell as technique T1059.001 and records its use by many criminal and state-sponsored groups [3]. The 2025 Threat Detection Report by Red Canary found that command and scripting interpreters were the most common attack technique for the sixth year in a row [4]. Removing PowerShell is not possible, because organisations depend on it; in 2022 the NSA and CISA advised organisations to keep PowerShell but to log and monitor it carefully [5].

Logging alone, however, only collects data. Someone still has to decide whether each script is safe or harmful, and do so quickly enough to stop an attack. Attackers make this harder by disguising their scripts with free tools such as Invoke-Obfuscation [6], and security teams already receive more alerts than they can check [7]. Machine learning can help by scoring every script automatically and supporting a fast and consistent response, in line with current incident-response guidance [8].

Problem statement. Given the text of a PowerShell script captured on a Windows computer, the system should (1) decide whether the script is malicious, with very few false alarms, and (2) automatically take a suitable action, such as raising an alert or blocking the script, while leaving the most serious actions to a human analyst.

2 Case Study: PowerShell in Real-World Attacks

This section looks at why attackers choose PowerShell, how a typical attack unfolds, and what four well-documented incidents teach defenders.

Why attackers choose PowerShell. PowerShell offers attackers several advantages at once. It is already installed and trusted, so running it does not look unusual. It can call almost any Windows function and .NET library, which allows it to download files, read passwords, change security settings and talk to remote servers. It can run code that exists only in memory, which leaves little evidence on disk for antivirus to scan. Finally, scripts can be encoded or disguised in many ways, so the same attack can look completely different each time [2], [6]. In earlier years, detailed PowerShell logging was also turned off by default in most organisations, so attackers could often act without leaving a record [2].

A typical attack chain. Figure 1 shows how PowerShell commonly appears in an attack. The attacker first gains entry, often through a phishing email with a malicious document. When the victim opens the document, a hidden macro starts PowerShell, which downloads the next stage of the attack (ATT&CK T1105) and runs it in memory (T1059.001, T1620). The attacker then sets up a way to come back later (persistence), steals passwords (T1003), turns off security tools (T1562) and spreads to other computers. The final goal may be data theft, financial fraud or ransomware. PowerShell can be involved at almost every stage, which means that monitoring it gives defenders many chances to stop the attack.

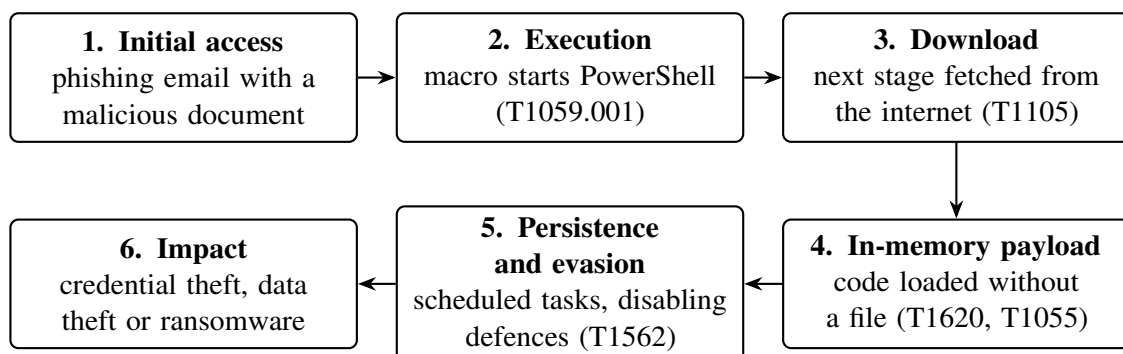


Figure 1: Typical stages of an attack that uses PowerShell, with MITRE ATT&CK technique IDs

Timeline of events. Figure 2 illustrates where the attacks mentioned in the following discussion stand in relation to a number of important defensive milestones. The attackers started using PowerShell early on, and the defenders have responded to this by doing things like improving logging and producing research tools and official guidelines. Scripting interpreters are still the most commonly seen method of attack [4].

Case 1: Odinaff attacks on financial organizations in 2016. Symantec reported in 2016 on a series of targeted attacks against financial institutions, including banks, securities, and

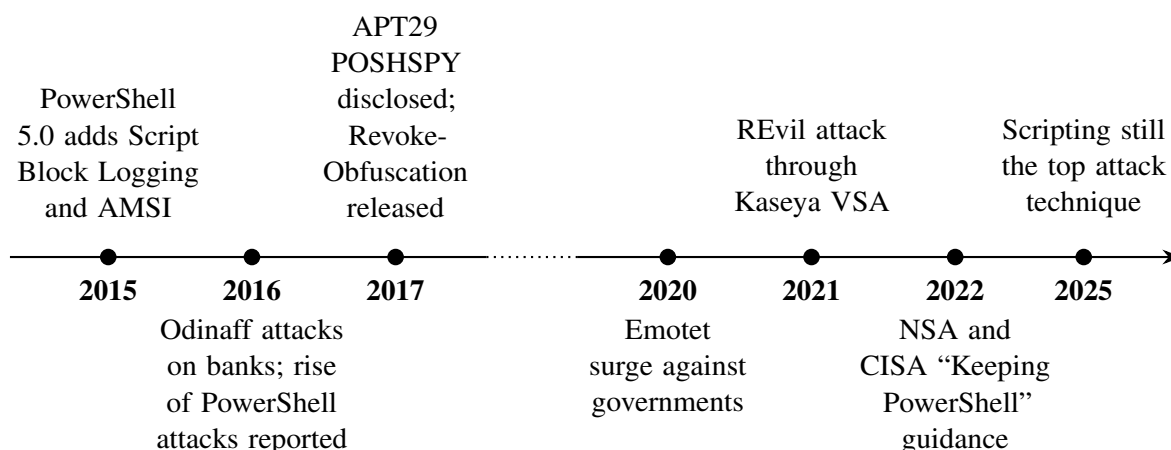


Figure 2: Timeline of major PowerShell-related attacks and defensive milestones [2], [4], [5], [9]– [13]

trading companies as well as payroll providers around the world, which were carried out using a Trojan dubbed Odinaff [9]. The Trojan was delivered using malicious Word macros, password-protected files, and existing botnets. Once inside the network, the hackers used lightweight hacking tools like Mimikatz along with legitimate utilities like PowerShell to explore the network and find important systems; some of the organizations affected were part of the SWIFT financial messaging network. Here, we see how cybercriminals leverage PowerShell alongside password stealing tools for stealthy network reconnaissance because the legitimate use of an administrative tool is, by itself, suspicious-free.

Case 2: APT29 and the POSHSPY backdoor (2015–2017). According to Mandiant, POSHSPY is a backdoor deployed by the state-sponsored APT29 in case their main tools got detected [10]. The POSHSPY malware was solely written in PowerShell and WMI. The encrypted PowerShell script was located in a custom WMI property and not in the file while the WMI event subscription ensured automatic restarts. This malware had the ability to download and execute other scripts and programs while connecting to its command and control servers rarely using legitimate local websites. Mandiant stated that because all processes used were legitimate, the malware could only be seen through enhanced logging or memory forensics. This is a clear example of why PowerShell logging is vital since without such logging, fileless malware can stay undetected for a very long time.

Case 3: Emotet malware campaigns 2014 onwards. The Emotet malware initially started as a banking Trojan and has become one of the most widespread malware families in the world, often serving as a vector for other malicious software, including ransomware. In October 2020, CISA warned of an increase in the number of Emotet phishing attacks targeted at state and local

governmental organizations, indicating that Emotet spreads inside the network via guessing passwords and replication onto the shared drives [12]. Many instances of Emotet campaigns used Word documents containing macros that ran an obfuscated PowerShell script, which retrieved the malware from a remote server. Analyzing PowerShell droppers used in the Emotet campaigns, Patsakis et al. discovered that they were highly obfuscated, and even the best language models could not reveal the download URLs of the hidden files [14].

Case 4: REvil ransomware through Kaseya VSA (2021). On 2 July 2021, the affiliate of REvil ransomware gang leveraged the vulnerabilities in Kaseya VSA, the software used for managing remote access by managed services providers, and released the malicious update to the clients’ computers, thereby affecting hundreds of companies in a single day [13]. Since the management agent was running at high privileges, the attackers used the PowerShell command to turn off Microsoft Defender real-time protection and script scanning prior to launching ransomware [15]. Thus, this example shows how fast modern attacks can spread – only one malicious update infected hundreds of companies in one day without much room left for manual analysis.

Table 1: Summary of the real-world cases

Case	Target	Use of PowerShell	Lesson for defenders
Odinaff (2016) [9]	Banks and financial firms	Moving through the network with Mimikatz and PowerShell	Legitimate tools hide malicious activity
APT29 POSHSPY (2015–2017) [10]	Undisclosed high-value organisations	Fileless backdoor stored in WMI	Enhanced logging is essential
Emotet (2014 onwards) [12], [14]	Businesses and governments	Obfuscated download command started by a macro	Signatures fail against obfuscation
REvil / Kaseya (2021) [13], [15]	Managed service providers and their clients	Disabling Microsoft Defender before ransomware	Response must be automatic and fast

Comparing the cases. Although the four incidents had different goals, they share a common pattern. The attackers were of very different types: a financially motivated criminal group (Odinaff), a state-sponsored espionage group (APT29), a large criminal malware operation (Emotet) and a ransomware affiliate (REvil). Their priorities also differed. APT29 valued stealth and long-term access, so POSHSPY was designed to run rarely and to blend in with normal

traffic. REvil valued speed and scale, so its PowerShell activity was short and direct, focused on disabling defences just before the ransomware ran. Emotet relied on volume, sending very large numbers of emails with disguised scripts, while Odinaff relied on careful exploration of a small number of selected networks. In every case, however, PowerShell was chosen because it was already present, trusted and capable, and in every case the malicious activity left traces in the scripts that ran. Mapped onto Figure 1, Emotet appears mainly at the execution and download stages, APT29 at the in-memory payload and persistence stages, Odinaff at the credential-theft and network-exploration stages, and REvil at the defence-evasion stage just before impact. A defensive system that examines every script, instead of looking for one particular tool or stage, therefore has a chance to interrupt very different kinds of attack.

Defensive controls suggested by the cases. Looking back at the four cases, several existing controls could have reduced their impact. Table 2 links common defensive controls, most of which are recommended in the joint NSA and CISA guidance [5], to the cases they would have helped with. The table also shows a gap: logging and scanning produce evidence, but none of these controls by itself decides quickly whether a particular script is malicious and what to do about it. This gap is the focus of the system developed in this study.

Table 2: Defensive controls and the cases they address

Control	What it does	Cases helped
Script Block Logging [16]	Records the code that actually runs, even after decoding	POSHSPY, Emotet, all others
AMSI scanning [17]	Lets antivirus check a script just before it runs	Emotet, POSHSPY
Constrained Language Mode with application control [5]	Limits what untrusted scripts are allowed to do	Odinaff, POSHSPY
Least privilege for management tools	Limits the damage if a remote management tool is abused	REvil / Kaseya
Protecting security settings	Stops scripts from switching off antivirus protection	REvil / Kaseya
ML-based detection with automated response (this study)	Scores every logged script and acts within seconds	All four cases

Lessons from the cases. Table 1 summarises the four cases. Together they lead to five requirements for a defensive system, which guide the rest of this study:

1. **Use the logs Windows already produces.** Script Block Logging and AMSI record the

actual code that runs, including code that was hidden or downloaded at run time [16], [17].

2. **Look at behaviour, not exact text.** Since scripts can easily be hidden, detection must depend on characteristics that will withstand any slight changes.
3. **Keep false alarms very low.** Admins use PowerShell for their day-to-day tasks; therefore, a detection system that inhibits the normal processes of admins will be regarded with suspicion.
4. **Respond automatically and quickly.** Ransomware spread is quick, calling for regular operations to be done without waiting for human assistance.
5. **Explain every decision.** The alerts must be set to reflect the purpose of the script being executed, e.g., based on ATT&CK [18] terminology.

3 Literature Review

Microsoft has added security features to PowerShell that make detection possible. *Script Block Logging* records the text of every script that runs as Windows Event ID 4104 [16], and the *Antimalware Scan Interface* (AMSI) allows antivirus software to scan a script just before it runs [17]. The MITRE ATT&CK framework provides a common language to describe attacker behaviour [18].

Because attackers disguise their scripts, much research has focused on obfuscation. Bohannon showed that a single command can be rewritten in many forms [6], and Bohannon and Holmes built Revoke-Obfuscation, which detects disguised scripts using statistical features [11]. Several tools try to remove the disguise before analysis, including PowerDrive [19], the method of Li et al. [20] and its extension by Xiong et al. [21], PowerDP [22] and PowerPeeler [23].

Machine learning has been applied to detect malicious PowerShell directly. Hendler et al. trained deep neural networks on more than 66,000 PowerShell commands and found that combining two different models gave the best results [24]. In later work they used AMSI data and reached a detection rate of nearly 90% with less than 0.1% false alarms [25], and Microsoft described using such models in its Defender products [26]. Rusak et al. used the syntax tree of scripts [27], Song et al. compared token-based and tree-based features and reported about 98% detection [28], and Mimura and Tajiri used word embeddings [29]. Fang et al. combined several types of features with a random forest, reported 98.93% accuracy and released the MPSD dataset

used in this study [30]. Saxe and Berlin showed that character-level models work well for short security texts such as web addresses [31], and general reviews of machine learning for malware are given by Ucci et al. [32] and Gibert et al. [33]. The newest work uses large language models to both detect and explain malicious PowerShell [34].

Researchers have also warned that machine-learning results in security can look better than they really are. Sommer and Paxson explained why intrusion detection is harder than ordinary classification [35]. Arp et al. listed common mistakes, such as allowing very similar samples to appear in both training and test data [36], and Pendlebury et al. showed that careless data splitting can make malware detectors appear much more accurate than they are [37]. Concerning the response aspect, SOAR technologies are used to bind the detection process with the action [38], while provenance-based techniques support analysts in dealing with a huge number of alerts [7], [39].

Table 3 is a list of some important studies. Most previous studies have focused solely on the detection aspect with highly accurate rates of performance. However, very few of them have analyzed duplicates within their datasets, and still fewer have incorporated detection with the response action.

Table 3: Summary of related work

Study	Method	Result
Hendler et al. [24]	Deep neural networks on PowerShell commands	High detection, very few false alarms
Hendler et al. [25]	AMSI data with pre-trained embeddings	About 90% detection
Bohannon & Holmes [11]	Statistical features to detect obfuscation	Robust obfuscation detection
Song et al. [28]	Token and syntax-tree features	About 98% detection
Fang et al. [30]	Hybrid features with random forest	98.93% accuracy
Wang et al. [34]	Large language models	Detection with explanations

4 Methodology

The proposed system follows the five requirements identified in the case study. It works in six steps, shown in Figure 3: scripts are collected from Windows logs, the data is cleaned, features are extracted, a machine-learning model gives each script a risk score between 0 and 1, a response policy chooses an action, and every decision is recorded.

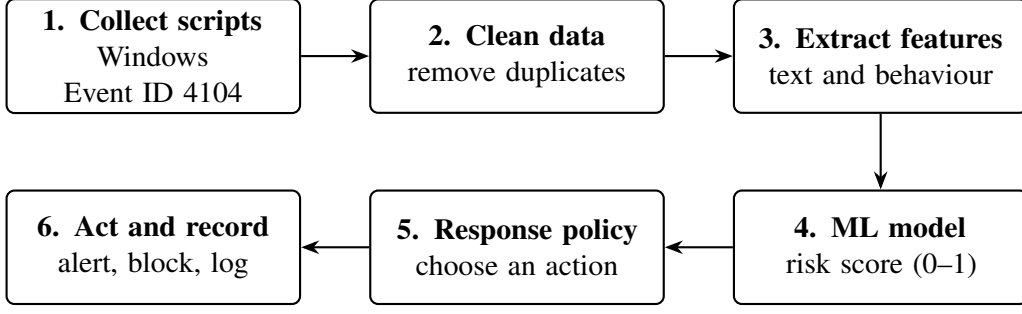


Figure 3: Workflow of the proposed system

Dataset. We used the public MPSD dataset released by Fang et al. [30]. It contains 4,202 malicious scripts, 4,316 benign scripts and 4,202 “mixed” scripts in which malicious code is hidden inside a normal script. The malicious scripts include downloaders, shellcode loaders, reverse shells and password-stealing tools, similar to those seen in the cases above. The benign scripts are normal administration scripts. All malicious scripts were handled only as text and were never run.

Data cleaning. Before training, the data was checked for problems, as recommended by Arp et al. [36]. Only 1,875 of the 4,202 malicious files were unique, which means 55.2% were exact copies; duplicates were removed. Twelve files appeared in both the malicious and benign folders and were removed because their true label is unknown. A special byte-order mark appeared at the start of about 35% of benign files but never in malicious files; since a model could learn this meaningless pattern, it was removed from all files. Finally, many malicious scripts differ only in variable names or numbers, so such near-copies were grouped into “families” (816 families for 1,875 malicious scripts), and a family was never split between training and testing. After cleaning, 1,875 malicious and 4,302 benign scripts remained.

Feature extraction. Each script is converted to lower case and the backtick character is removed, because PowerShell ignores both and attackers use them to disguise commands. Two kinds of features are then extracted. The first are *text features*: the script is split into short sequences of 3 to 5 characters, and each sequence is weighted with TF-IDF [40], so that a sequence that is frequent in one script but rare in others receives a high weight:

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}, \quad (1)$$

where $\text{tf}(t, d)$ is how often sequence t appears in script d , N is the number of scripts and $\text{df}(t)$ is the number of scripts containing t . The second are 33 *behaviour and statistical features*,

such as script length, number of lines, randomness of characters and counts of suspicious keywords. The keywords are grouped by attacker behaviour and linked to ATT&CK, for example downloading tools (T1105), running code (T1059.001), code injection (T1055) and disabling defences (T1562), which are the same behaviours seen in the case study.

Machine-learning models. Four models and one ensemble were trained with the scikit-learn library [41]: logistic regression (LR) on the text features, LR on the behaviour features as a simple baseline, a random forest (RF) of 300 decision trees [42], and a gradient boosting model (HGB) similar to LightGBM [43]. The ensemble averages the scores of the text-based LR and the behaviour-based RF; because the two look at different evidence, combining them makes the result more stable. Deep learning was not used because the dataset is small and simpler models are faster and easier to explain.

Evaluation. The models were tested with five-fold cross-validation [44]: the data is split into five parts, and each part is used once for testing while the other four are used for training. Families of near-copies were always kept in the same part, so a model is never tested on a script it has almost seen before. With TP, FP, TN and FN denoting true positives, false positives, true negatives and false negatives, the main measures were:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2)$$

Accuracy and the area under the ROC curve (AUC) were also reported.

Automated response. The risk score is used to choose one of four actions (Table 4). Less harmful actions are taken automatically, while the most serious action, disconnecting a computer from the network, needs a human analyst’s approval. Every decision is saved in a log together with the ATT&CK techniques found, and suspicious web addresses are written in a safe form (for example, hxxp instead of http).

Table 4: Response levels used by the system

Level	Risk score	Action
Allow	below 0.30	Record the result only
Alert	0.30 to 0.70	Send an alert to the security team with details
Contain	0.70 or above	Stop the script, quarantine the file and block its web addresses
Isolate	0.95 or above, with dangerous behaviour	Disconnect the computer from the network after analyst approval

5 Implementation Details

The system was written in Python 3.13 using scikit-learn for machine learning, pandas and NumPy for data handling and Matplotlib for graphs, and it was run on an ordinary computer with a 4-core processor and 15 GB of memory, without a GPU. This program consists of four modules: data, which cleanses data set; features, which calculates textual and behavioral features; model, which trains and tests models; and response modules. Response module gets input from Windows Event ID 4104 data and calculates risk score from model, response level from this risk score, related ATT&CK technique and makes record in log. For security reasons, this module runs in “dry-run” mode, where actions are simulated but not performed. In a real business environment, Script Block Logging would be enabled by Group Policy [16]; logs will be transferred to the central server for risk scoring, and the response module will interact with company’s security infrastructure. Seven unit tests check the key functionality, and all the tests pass successfully.

6 Results and Discussion

Model performance. The results obtained from all the models are shown in Table 5, while the F1-scores are compared in Figure 4. All models performed very well, showing that it is easy to tell the difference between a bad and good PowerShell script. The best performing model among the four models is the text-based logistic regression (LR) model with an F1-score of 0.974; the ensemble model had the almost similar F1-score of 0.972 but had fewer false positives (30 vs 37). Since false positives may interfere with the administration activities, and that is the third criterion of the case study, we opted for the ensemble.

Table 5: Performance of the models (five-fold cross-validation)

Model	Accuracy	Precision	Recall	F1	AUC
LR (behaviour)	0.974	0.953	0.960	0.957	0.988
RF (behaviour)	0.980	0.982	0.952	0.967	0.995
HGB (behaviour)	0.979	0.984	0.948	0.966	0.996
LR (text, TF-IDF)	0.985	0.980	0.969	0.974	0.999
Ensemble (LR + RF)	0.984	0.984	0.962	0.972	0.998

The confusion matrix for the ensemble is illustrated in Figure 5. Among the 4,302 benign scripts, 4,272 (99.3%) were accurately identified, while among the 1,875 malicious scripts,

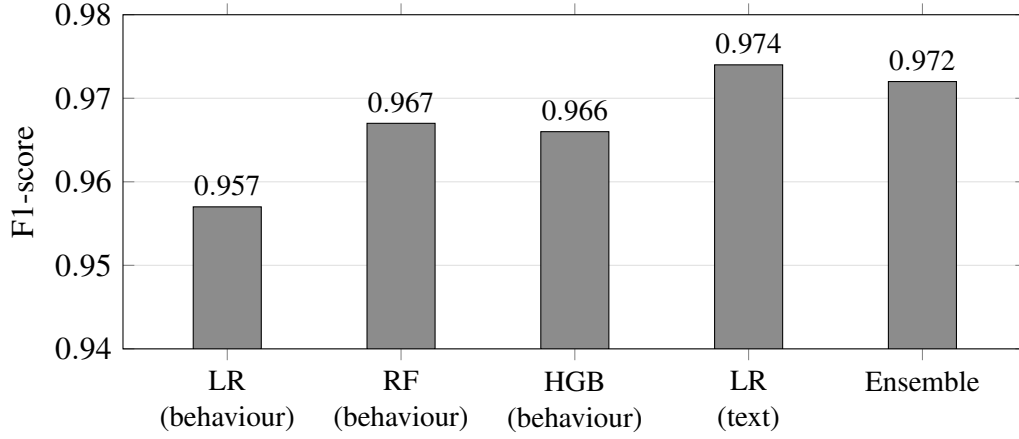


Figure 4: F1-score of each model

1,803 (96.2%) were detected correctly.

		Predicted	
		Benign	Malicious
Actual	Benign	4,272 (99.3%)	30 (0.7%)
	Malicious	72 (3.8%)	1,803 (96.2%)

Figure 5: Confusion matrix of the ensemble model

Effect of eliminating duplicate records. Upon running the ensemble on data containing duplicate records with a simple random partition, the F1-score of the algorithm increased to 0.988, and the percentage of missed malicious scripts decreased to 1.7%. This increase in the value can be deceiving, since the algorithm was partially trained on scripts that it was supposed to encounter again in the future. In this case, without cleansing, the actual number of attacks missed would be overestimated by more than half, in accordance with the remarks by Arp et al. [36] and Pendlebury et al. [37].

Malicious code hidden inside normal scripts. Malicious code that is embedded within an otherwise legitimate script. This problem became obvious in testing the models on the “mixed” scripts, which have a malicious code embedded into a long legitimate script, similar to the Emotet’s “dropper” disguise. This became the primary vulnerability of the ensemble, which could identify 46.7% of such cases. The reason was that there was a lot of legitimate code covering up for the relatively small malicious part of it. Adding examples of mixed scripts to the training dataset increased detection rate up to 96.4%, while causing only a slight increase in

false positives (0.70% to 1.05%).

Automated response results. To test the complete system, the ensemble was trained on four parts of the data, and the fifth part (1,233 scripts it had never seen) was sent through the response system as Windows log events. Table 6 shows the outcome. Most malicious scripts were stopped automatically, very few benign scripts were blocked, and a typical script was processed in about 91 milliseconds, which is fast enough to act before an attack like the REvil case can spread. The most common attacker behaviours found were running code with PowerShell (T1059.001), loading code into memory (T1620), code injection (T1055) and downloading tools (T1105), the same stages shown in Figure 1.

Table 6: Results of the automated response test

Measure	Result
Scripts tested (malicious / benign)	373 / 860
Malicious scripts stopped automatically	91.2%
Malicious scripts at least alerted	97.1%
Benign scripts wrongly blocked	0.23% (2 scripts)
Benign scripts sent as alerts	0.81%
Typical time per script (median)	90.7 ms

Relating the results to the cases. Table 7 explains how the system is designed to react to the PowerShell activity seen in the four cases. These are expected behaviours based on the features and response rules, not results of tests on the original incident data, which is not publicly available.

Table 7: Expected reaction of the system to the PowerShell activity in each case

Case	Signals the system looks for	Expected response
Odinaff	PowerShell used with credential-theft tools such as Mimikatz (T1003)	High risk; Contain, or Isolate with analyst approval
APT29 POSHSPY	Downloading and running further code, reflection and encryption (T1105, T1620)	Logged through Event ID 4104 and scored; Alert or Contain
Emotet	Encoded download command started from a document (T1027, T1105)	Contain; download address blocked
REvil / Kaseya	Command switching off Microsoft Defender (T1562)	Defence-evasion signal raised; Alert or Contain before ransomware runs

Illustrative walkthrough: an Emotet-style infection. To show how the parts of the system

work together, consider an employee who opens an infected invoice document, as in the Emotet campaigns. The following sequence describes the intended behaviour of the system; it is an illustration based on its design, not a recorded incident.

1. In the document, the macro invokes PowerShell to execute the command that is encoded. When executed, Windows decodes the command and logs the decoded script as event id 4104.
2. It takes the event after seconds and then analyzes its features. Both encoding (T1027) and downloading (T1105) indicators are available, and the textual features are similar to those of the known downloaders, resulting in a high risk score.
3. Since the value is greater than 0.70, the script becomes contained as the powershell process is terminated, the file is quarantined and its download location blocked, ensuring that the next stage of the malware never downloads.
4. The security team will receive an alert that will contain the score, the ATT&CK technique used, and the blocked address. Should the presence of code injection and credential theft have been observed with an extremely high score, then computer isolation will be recommended, pending the analyst's approval.

This walk-through highlights how the lessons learned from the case study (use current logs, consider the behavior, react fast, and explain each decision) can be seen in the system's architecture.

Limitations. Some limitations exist in the research. It uses just one publicly available database; therefore, further verification is required by performing tests using the actual data of enterprises. The proposed solution analyzes only the text of the script and not additional information about it, such as which application runs this script, which may be useful in cases like Odinaff, when legal software is misused.

This case study explored the application of PowerShell by attackers and the possibilities of using machine learning in order to prevent the corresponding cyber attacks. Four cases – Odinaff, APT29, Emotet and REvil – demonstrated how attackers exploit PowerShell at all stages of attacks, disguise the scripts in order to make them signature-less, and do significant harm within seconds. Based on this analysis, a system which ranks PowerShell scripts based on Windows logs and automatically provides a response was developed. As a result of removing duplicates and mislabeled data from the MPSD dataset, an ensemble of a text-based logistic regression and a behavior-based random forest achieved the F1-score of 0.972 and detected

96.2% of malicious scripts while producing false alarms only in 0.7%. For an out-of-sample test, the response system managed to prevent 91.2% of malicious scripts, made mistakes blocking 0.23% of benign scripts and required 91 milliseconds per one script. Two more lessons were obtained. First, it was found that duplicate detection is required when working with the dataset to avoid the illusion of high-quality results. Second, malicious scripts embedded into the legitimate ones require special training samples. For future research, the following steps are recommended. The system can be applied to company logs, integrated with AMSI in order to prevent execution of suspicious scripts, provided with data about currently running processes and combined with the large language models providing explanations for the alerts [34]. Thus, explainable machine learning combined with proper response policy constitutes the efficient defense against PowerShell attacks which correspond to expert recommendations to control PowerShell [5].

References

- [1] C. Wueest and H. Anand, “Living off the land and fileless attack techniques,” Symantec Internet Security Threat Report (ISTR) Special Report, Symantec Corp., Jul. 2017.
- [2] C. Wueest, “The increased use of PowerShell in attacks,” Symantec Security Response White Paper, Symantec Corp., Oct. 2016.
- [3] The MITRE Corporation, “Command and Scripting Interpreter: PowerShell (T1059.001),” *MITRE ATT&CK*. [Online]. Available: <https://attack.mitre.org/techniques/T1059/001/>
- [4] Red Canary, “2025 Threat Detection Report,” Red Canary Inc., Denver, CO, USA, 2025. [Online]. Available: <https://redcanary.com/threat-detection-report/>
- [5] National Security Agency, Cybersecurity and Infrastructure Security Agency, New Zealand National Cyber Security Centre and UK National Cyber Security Centre, “Keeping PowerShell: Security measures to use and embrace,” Joint Cybersecurity Information Sheet, Jun. 2022.
- [6] D. Bohannon, “Invoke-Obfuscation: PowerShell command and script obfuscator,” presented at DerbyCon 6.0, Louisville, KY, USA, 2016. [Online]. Available: <https://github.com/danielbohannon/Invoke-Obfuscation>
- [7] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li and A. Bates, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2019.
- [8] A. Nelson, S. Rekhi, M. Souppaya and K. Scarfone, “Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile,”

NIST Special Publication 800-61 Rev. 3, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2025.

- [9] Symantec Security Response, “Odinaff: New Trojan used in high level financial attacks,” Symantec Threat Intelligence Blog, Oct. 11, 2016. [Online]. Available: <https://www.security.com/threat-intelligence/odinaff-trojan-financial-attacks>
- [10] M. Dunwoody, “Dissecting one of APT29’s fileless WMI and PowerShell backdoors (POSHSPY),” Mandiant (FireEye) Threat Research Blog, Apr. 3, 2017. [Online]. Available: <https://cloud.google.com/blog/topics/threat-intelligence/dissecting-one-ofap/>
- [11] D. Bohannon and L. Holmes, “Revoke-Obfuscation: PowerShell obfuscation detection using science,” White Paper, Black Hat USA, Las Vegas, NV, USA, Jul. 2017.
- [12] Cybersecurity and Infrastructure Security Agency and Multi-State Information Sharing and Analysis Center, “Emotet malware,” Cybersecurity Alert AA20-280A, Oct. 6, 2020. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-280a>
- [13] Sophos, “Independence Day: REvil uses supply chain exploit to attack hundreds of businesses,” Sophos News, Jul. 2021. [Online]. Available: <https://www.sophos.com/en-us/blog/independence-day-revil-uses-supply-chain-exploit-to-attack-hundreds-of-businesses>
- [14] C. Patsakis, F. Casino and N. Lykousas, “Assessing LLMs in malicious code deobfuscation of real-world malware campaigns,” *Expert Systems with Applications*, vol. 256, Art. no. 124912, 2024.
- [15] Splunk Threat Research Team, “Kaseya, Sera. What REvil shall encrypt, shall encrypt,” Splunk Blog, Jul. 2021. [Online]. Available: https://www.splunk.com/en_us/blog/security/kaseya-sera-what-revil-shall-encrypt-shall-encrypt.html
- [16] Microsoft, “about_Logging_Windows,” PowerShell Documentation, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows
- [17] Microsoft, “Antimalware Scan Interface (AMSI),” Win32 Apps Documentation, Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>
- [18] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, McLean, VA, USA, Tech. Rep. MTR180360, 2018.

- [19] D. Ugarte, D. Maiorca, F. Cara and G. Giacinto, “PowerDrive: Accurate de-obfuscation and analysis of PowerShell malware,” in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2019)*, Lecture Notes in Computer Science, vol. 11543. Cham, Switzerland: Springer, 2019, pp. 240–259.
- [20] Z. Li, Q. A. Chen, C. Xiong, Y. Chen, T. Zhu and H. Yang, “Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, London, UK, 2019, pp. 1831–1847.
- [21] C. Xiong, Z. Li, Y. Chen, T. Zhu, J. Wang, H. Yang and W. Ruan, “Generic, efficient, and effective deobfuscation and semantic-aware attack detection for PowerShell scripts,” *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 3, pp. 361–381, 2022.
- [22] M.-H. Tsai, C.-C. Lin, Z.-G. He, W.-C. Yang and C.-L. Lei, “PowerDP: De-obfuscating and profiling malicious PowerShell commands with multi-label classifiers,” *IEEE Access*, vol. 11, 2023.
- [23] R. Li, C. Zhang, H. Chai, L. Ying, H. Duan and J. Tao, “PowerPeeler: A precise and general dynamic deobfuscation method for PowerShell scripts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS ’24)*, Salt Lake City, UT, USA, 2024.
- [24] D. Hendler, S. Kels and A. Rubin, “Detecting malicious PowerShell commands using deep neural networks,” in *Proc. ACM Asia Conf. Computer and Communications Security (ASIACCS)*, Incheon, Republic of Korea, 2018, pp. 187–197.
- [25] D. Hendler, S. Kels and A. Rubin, “AMSI-based detection of malicious PowerShell code using contextual embeddings,” in *Proc. 15th ACM Asia Conf. Computer and Communications Security (ASIA CCS ’20)*, Taipei, Taiwan, 2020, pp. 679–693.
- [26] Microsoft Defender Research Team, “Deep learning rises: New methods for detecting malicious PowerShell,” Microsoft Security Blog, Sep. 3, 2019. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2019/09/03/deep-learning-rises-new-methods-for-detecting-malicious-powershell/>
- [27] G. Rusak, A. Al-Dujaili and U.-M. O’Reilly, “AST-based deep learning for detecting malicious PowerShell,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Toronto, ON, Canada, 2018, pp. 2276–2278.
- [28] J. Song, J. Kim, S. Choi, J. Kim and I. Kim, “Evaluations of AI-based malicious PowerShell detection with feature optimizations,” *ETRI Journal*, vol. 43, no. 3, pp. 549–560, 2021.
- [29] M. Mimura and Y. Tajiri, “Static detection of malicious PowerShell based on word embeddings,” *Internet of Things*, vol. 15, Art. no. 100404, 2021.
- [30] Y. Fang, X. Zhou and C. Huang, “Effective method for detecting malicious PowerShell scripts based on hybrid features,” *Neurocomputing*, vol. 448, pp. 30–39, 2021.

- [31] J. Saxe and K. Berlin, “eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys,” arXiv preprint arXiv:1702.08568, 2017.
- [32] D. Ucci, L. Aniello and R. Baldoni, “Survey of machine learning techniques for malware analysis,” *Computers & Security*, vol. 81, pp. 123–147, 2019.
- [33] D. Gibert, C. Mateu and J. Planes, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” *Journal of Network and Computer Applications*, vol. 153, Art. no. 102526, 2020.
- [34] M. Wang, E. Topolovec, B. Arana, N. V. P. S. G. C. Kuladeep, Z. Xi, G. Yan, X. Shu, S. D. Stoller and P. Yang, “Detection and explanation of PowerShell malware with large language models,” Manuscript, Binghamton University, IBM T. J. Watson Research Center and Stony Brook University, 2026.
- [35] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.
- [36] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.
- [37] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, 2019, pp. 729–746.
- [38] C. Islam, M. A. Babar and S. Nepal, “A multi-vocal review of security orchestration,” *ACM Computing Surveys*, vol. 52, no. 2, Art. no. 37, 2019.
- [39] W. U. Hassan, A. Bates and D. Marino, “Tactical provenance analysis for endpoint detection and response systems,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, San Francisco, CA, USA, 2020, pp. 1172–1189.
- [40] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [41] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [42] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [43] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154.
- [44] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Montreal, QC, Canada, 1995, pp. 1137–1143.